UNIT 2 · TOPIC 2.2

Physical Security Breaches

Assignment

Instructions: 16 practice questions + 3 coding assignment(s), 29 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

2.2.a · Unauthorized physical access (tailgating, piggybacking, lock bypass)

1. An attacker carries boxes to appear busy and follows an employee through a badge-reader door without swiping. This is BEST described as: (1 pt)

- ○ A. Piggybacking
- ○ B. Tailgating
- ○ C. RFID cloning
- ○ D. Bump keying

2. Which physical attack leaves evidence in a badge-access log that looks identical to a legitimate single entry? (1 pt)

- ○ A. Lock picking
- ○ B. Bump keying
- ○ C. Tailgating
- ○ D. RFID jamming

3. Select ALL techniques that can grant physical access WITHOUT the correct badge credential. (3 pts)

- □ A. Tailgating
- □ B. RFID badge cloning
- □ C. Multi-factor authentication
- □ D. Lock picking

(Select ALL that apply.)

4. A building access log shows badge ID 1042 swiping door D1 at 09:00:02 and door D2 (40 meters away) at 09:00:05. Why is this record suspicious? (1 pt)

5. Older 125 kHz HID proximity cards are especially vulnerable to which attack? (1 pt)

- ○ A. Bump keying
- ○ B. RFID cloning
- ○ C. Lock shimming
- ○ D. Tailgating

6. Name the physical security control that uses two interlocked doors to prevent a second person from following through a secured entry. (1 pt)

2.2.b · Environmental vulnerabilities (power, HVAC, fire, water)

7. Why do data centers use clean-agent fire suppression (e.g., FM-200, inert gas) instead of water sprinklers? (1 pt)

- ○ A. Water sprinklers would themselves destroy the electronics they are meant to protect
- ○ B. Clean-agent systems are cheaper to install
- ○ C. Clean-agent systems put out fires faster than water in all cases
- ○ D. Fire codes prohibit water sprinklers in any commercial building

8. A server room's temperature sensor and humidity sensor both cross their warning thresholds at the same time. What does this combined pattern most likely indicate? (1 pt)

- ○ A. A single faulty sensor
- ○ B. A tailgating attempt
- ○ C. A failing or overloaded HVAC (cooling) system
- ○ D. A power surge

9. Which TWO controls directly address power-related environmental risk? Select all that apply. (2 pts)

- □ A. Uninterruptible power supply (UPS)
- □ B. Clean-agent fire suppression
- □ C. Backup generator with N+1 redundancy
- □ D. Raised flooring

(Select ALL that apply.)

10. Why is raised flooring used as a control against water/flood risk in a data center? (1 pt)

11. Why should environmental risks (power, HVAC, fire, water) be analyzed with the same rigor as a targeted cyberattack? (1 pt)

- ○ A. They are legally required to be reported the same way
- ○ B. They can produce the same availability or integrity loss as a targeted attack, even without an adversary
- ○ C. They are actually more common than cyberattacks in every organization
- ○ D. Insurance companies require it

2.2.c · Device theft and tampering

12. What is the single most effective control against data exposure from a STOLEN laptop? (1 pt)

- ○ A. A strong login password
- ○ B. Full-disk encryption
- ○ C. A Kensington cable lock
- ○ D. An asset tag

13. What distinguishes an evil-maid attack from ordinary device theft? (1 pt)

- ○ A. The device is covertly tampered with and returned, so the owner does not know anything happened
- ○ B. The attacker copies the data and keeps the device, so the owner notices it is gone
- ○ C. The attacker uses a stolen password instead of physical access
- ○ D. The device is damaged so it cannot be used, which is an availability attack

14. Which TWO of the following are physical/behavioral signs that a device tampering incident may have occurred? Select all that apply. (2 pts)

- □ A. A tamper-evident seal shows a torn or VOID pattern
- □ B. The device's login password was recently changed by the owner
- □ C. An unfamiliar dongle is found plugged into the keyboard port
- □ D. The device was rebooted after a routine software update

(Select ALL that apply.)

15. Why can a tampering incident still be a risk even if the tamper-evident seal is intact? (1 pt)

16. A stolen laptop was encrypted, contained sensitive data, and was not reported missing for 60 hours. Per this atom's triage rules, what risk level should this incident receive? (1 pt)

- ○ A. LOW
- ○ B. HIGH
- ○ C. MEDIUM
- ○ D. Cannot be determined

Coding Assignments (Python 3)

2.2.a · Unauthorized physical access (tailgating, piggybacking, lock bypass)

3 pts · auto-graded on the site

Coding problem 2.2.a: Detect unauthorized physical access in a badge log

Skill: Analyze Risk · **Maps to:** 2.2.A, 2.2.B · **Language:** Python 3 (pytest)

Problem

A building security system writes one log record per door event. Your job is to implement the analyst's first-pass filter: given a list of log records, return the event IDs of all records that indicate a likely unauthorized physical access attempt, based on two detection rules:

Rule 1, Impossible speed (tailgating / badge pass-back indicator). If the same badge ID appears at two different doors and the time between events is less than the minimum walk time (distance_meters / MAX_WALK_SPEED), the second event is suspicious. `MAX_WALK_SPEED = 2.0` meters per second.

Rule 2, Sensor mismatch (tailgating indicator). If an event has `"sensor_count": 2` (or higher) but only one badge swipe was recorded (i.e., the event itself has `"badge": <a badge id>` and no second badge event at the same door within 1 second), that event is suspicious.

Return the list of flagged event IDs (strings), sorted ascending. An event may match both rules but should appear only once.

Function contract

Implement `solve(records: list[dict]) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`records`, a list of dicts, each with keys:

- `"id"` (str): unique event identifier, e.g. `"E001"`
- `"time"` (int): Unix timestamp in seconds (integer)
- `"badge"` (str | None): badge ID that was swiped, or None if no swipe
- `"door"` (str): door identifier, e.g. `"D1"`
- `"distance_from_prev"` (int | None): meters from the previous door this badge was seen at, or None if this is the badge's first appearance or badge is None
- `"sensor_count"` (int): number of bodies the door sensor detected crossing (1 = normal)

Records are provided in chronological order (ascending `"time"`).

Output

A sorted `list[str]` of flagged event IDs. Empty list if no violations found.

Examples

```
[
  {"id":"E001","time":1000,"badge":"B-204","door":"D1","distance_from_prev":None,"sensor_count":1},
  {"id":"E002","time":1003,"badge":"B-204","door":"D2","distance_from_prev":50,"sensor_count":1},
  {"id":"E003","time":2000,"badge":"B-117","door":"D1","distance_from_prev":None,"sensor_count":2},
]
-> ["E002", "E003"]

E002: B-204 covers 50 m in 3 s = 16.7 m/s > 2.0 m/s  (Rule 1)
E003: sensor_count=2 with one swipe                   (Rule 2)
```

Constraints

- Deterministic, offline, standard library only.
- MAX_WALK_SPEED = 2.0 m/s (defined in interface.md).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 2.2.a. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List, Dict, Any

MAX_WALK_SPEED = 2.0  # meters per second


def solve(records: List[Dict[str, Any]]) -> List[str]:
    """Return sorted list of event IDs that indicate unauthorized physical access.
```

```
    Two detection rules (see prompt.md and interface.md):
      Rule 1 — Impossible speed: a badge appears at a second door faster than
               MAX_WALK_SPEED (2.0 m/s) allows given the distance.
      Rule 2 — Sensor mismatch: sensor_count >= 2 with a single badge swipe.

    Args:
        records: list of dicts with keys id, time, badge, door,
                 distance_from_prev, sensor_count (chronological order).
    Returns:
        Sorted, de-duplicated list of flagged event IDs.
    """
    # TODO: implement both detection rules and return sorted flagged IDs.
    raise NotImplementedError("implement solve() — see prompt.md")
```

2.2.b · Environmental vulnerabilities (power, HVAC, fire, water)

3 pts · auto-graded on the site

Coding problem 2.2.b: Environmental sensor triage

Skill: Analyze Risk · **Maps to:** 2.2.A · **Language:** Python 3 (pytest)

Problem

A data center streams readings from environmental sensors placed around each server room: power (voltage as a percent of nominal), temperature (Celsius), humidity (percent relative humidity), smoke detectors, and water/flood detectors. Facilities staff need every reading automatically triaged into a severity level so a human only has to look at what matters.

Your task is to write a function `solve(data)` that takes a list of sensor readings and returns a dictionary mapping each reading's `id` to a severity string: `"NORMAL"`, `"WARNING"`, or `"CRITICAL"`.

Per-reading thresholds (by `type`)

`power` (`value` = supply voltage as a percent of nominal, 100 = nominal):

- `CRITICAL` if `value == 0` (total outage) or `value < 90` or `value > 110`
- `WARNING` if `90 <= value < 95` or `105 < value <= 110`
- `NORMAL` otherwise (`95 <= value <= 105`)

`temperature` (`value` = degrees Celsius):

- `CRITICAL` if `value > 30`
- `WARNING` if `27 < value <= 30` or `value < 15`
- `NORMAL` otherwise (`15 <= value <= 27`)

`humidity` (`value` = percent relative humidity):

- `CRITICAL` if `value > 70` or `value < 20`
- `WARNING` if `60 < value <= 70` or `20 <= value < 40`
- `NORMAL` otherwise (`40 <= value <= 60`)

`smoke` (`value` = detector reading, `0` = clear, any positive value = smoke detected):

- `CRITICAL` if `value > 0`
- `NORMAL` otherwise

`water` (`value` = detector reading, `0` = dry, any positive value = moisture detected):

- `CRITICAL` if `value > 0`
- `NORMAL` otherwise

Combined-signal escalation (cooling-failure pattern) After computing each reading's per-reading severity above, apply one escalation rule: for readings that share the same `room`, if there is **both** a `temperature` reading rated `WARNING` or `CRITICAL` **and** a `humidity` reading rated `WARNING` or `CRITICAL` in that same room, escalate *both* of those specific readings (temperature and humidity only) to `CRITICAL`. This models a failing HVAC unit, where rising temperature and drifting humidity together are a stronger signal than either alone. Other reading types in the room (power, smoke, water) are not affected by this rule, and a room where only one of the two (temperature or humidity) is elevated is not escalated.

Function contract

Implement `solve(data: list[dict]) -> dict[str, str]` in `starter.py`.

Input

A list of dictionaries, each representing one sensor reading:

```
[
    {"id": "s1", "room": "server-room-a", "type": "temperature", "value": 29},
    {"id": "s2", "room": "server-room-a", "type": "humidity", "value": 65},
    {"id": "s3", "room": "server-room-b", "type": "power", "value": 100},
]
```

Output

A dictionary mapping each reading's `"id"` to its severity string:

```
{
    "s1": "CRITICAL",   # escalated: room also had a humidity WARNING/CRITICAL
    "s2": "CRITICAL",   # escalated: room also had a temperature WARNING/CRITICAL
    "s3": "NORMAL",
}
```

Constraints

- `type` is one of `"power"`, `"temperature"`, `"humidity"`, `"smoke"`, `"water"`.
- `value` is a non-negative number (float or int) except `power`, which may be `0` (outage).
- Pure function, deterministic, standard library only.

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 2.2.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Any


def solve(data: list[dict]) -> dict[str, str]:
    """Triage a list of environmental sensor readings into severity levels.

    Args:
        data: A list of dicts, each with keys id, room, type, value.
    Returns:
```

```
        A dictionary mapping reading ids to "NORMAL" | "WARNING" | "CRITICAL".
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

2.2.c · Device theft and tampering

3 pts · auto-graded on the site

Coding problem 2.2.c: Device theft/tampering incident triage

Skill: Analyze Risk · **Maps to:** 2.2.B, 2.2.C · **Language:** Python 3 (pytest)

Problem

The IT security team receives reports whenever a device is reported stolen or found to show signs of tampering. Each report needs to be triaged into a risk level so the team knows what needs an immediate incident response versus what can be logged and monitored.

Your task is to write a function `solve(data)` that takes a list of incident reports and returns a dictionary mapping each incident's `id` to a risk level: `"LOW"`, `"MEDIUM"`, or `"HIGH"`.

Rules, by incident `type`

`type == "theft"` (a device was physically taken): 1. `"HIGH"` if the device was **not** encrypted (`encrypted` is `False`), an unencrypted stolen device exposes its data with no cryptographic barrier. 2. `"MEDIUM"` if the device **was** encrypted, but `contains_sensitive_data` is `True` **and** `hours_since_check` is greater than 48, encryption protects the data, but a long detection delay on a sensitive device still raises risk (e.g., the device may have been unlocked and in active use when taken). 3. `"LOW"` otherwise.

`type == "tampering"` (a device was found to show signs of covert alteration): 1. `"HIGH"` if the tamper-evident seal was **not** intact (`seal_intact` is `False`), physical evidence the enclosure was opened. 2. `"MEDIUM"` if the seal **was** intact but `hours_since_check` is greater than 48, a long window since the last inventory check means tampering that does not require opening the enclosure (e.g., a rogue USB implant) could have gone unnoticed even with an intact seal. 3. `"LOW"` otherwise.

Any other `type` value should be treated as `"LOW"`.

Function contract

Implement `solve(data: list[dict]) -> dict[str, str]` in `starter.py`.

Input

A list of dictionaries, each representing one incident report:

```
[
    {
        "id": "inc-1",
        "type": "theft",
        "encrypted": False,
        "seal_intact": True,
        "contains_sensitive_data": True,
        "hours_since_check": 10,
    },
    {
        "id": "inc-2",
        "type": "tampering",
        "encrypted": True,
        "seal_intact": False,
        "contains_sensitive_data": False,
        "hours_since_check": 5,
    },
]
```

Every report includes all five fields (`type`, `encrypted`, `seal_intact`, `contains_sensitive_data`, `hours_since_check`), even if a field is not relevant to that report's `type` (e.g., `seal_intact` is present but ignored for `type == "theft"`).

Output

A dictionary mapping each incident's `"id"` to its risk level string:

```
{
    "inc-1": "HIGH",
    "inc-2": "HIGH",
}
```

Constraints

- `type` is `"theft"` or `"tampering"` (or, in hidden tests, possibly another string that must default to `"LOW"`).
- `encrypted`, `seal_intact`, `contains_sensitive_data` are booleans.
- `hours_since_check` is a non-negative number (int or float).
- Pure function, deterministic, standard library only.

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 2.2.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Any


def solve(data: list[dict]) -> dict[str, str]:
    """Triage device theft/tampering incident reports into risk levels.

    Args:
        data: A list of dicts, each with keys id, type, encrypted, seal_intact,
            contains_sensitive_data, hours_since_check.
    Returns:
        A dictionary mapping incident ids to "LOW" | "MEDIUM" | "HIGH".
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```